

以 R 語言控制瀏覽器

從網頁基礎到台灣高鐵自動訂票

網頁基礎：HTML

瀏覽器打開網頁時發生了什麼

你在 Chrome 網址列輸入一個網址，按下 Enter：

- ❶ 瀏覽器透過網路向**伺服器**要一份檔案
- ❷ 伺服器回傳一份**純文字**檔案（.html）
- ❸ 瀏覽器**解析**這份文字，畫出你看到的頁面

這份純文字檔案，就是 **HTML**。

你看到的按鈕、輸入框、下拉選單，全都是 HTML 標籤。

- **HTML** = HyperText Markup Language (超文本標記語言)
- 一種用**標籤 (tag)** 來描述文件結構的純文字格式

和 Word 的差異：

	Word	HTML
粗體	選取文字 → 按 B	<code></code> 文字 <code></code>
段落	按 Enter 換行	<code><p></code> 文字 <code></p></code>
格式存在哪	二進位檔案內部	直接寫在文字裡

一份最小的 HTML 文件

```
<!DOCTYPE html>
<html>
  <head>
    <title>我的第一個網頁</title>
  </head>
  <body>
    <p>Hello, world!</p>
  </body>
</html>
```

標籤	意義
<!DOCTYPE html>	宣告這是 HTML5 文件
<html>	整份文件的根（最外層）
<head>	後設資料（標題、編碼等，不顯示在頁面上）
<body>	頁面上 看得到的內容 都在這裡

- ❶ **成對出現**：開始標籤 `<p>` + 結束標籤 `</p>`（多一個 `/`）

```
<p>這是一個段落。</p>
```

- ❷ **可以嵌套**（標籤裡放標籤）：

```
<div>  
  <p>段落在 div 裡面</p>  
</div>
```

- ❸ **有些標籤自關閉**（沒有內容、不需要結束標籤）：

```
  
<input type="text">  
<br>
```

```
<!-- h1 最大，h6 最小 -->
```

```
<h1>主標題</h1>
```

```
<h2>副標題</h2>
```

```
<h3>小節標題</h3>
```

```
<!-- 段落 -->
```

```
<p>這是一個段落文字。</p>
```

```
<!-- 粗體與斜體 -->
```

```
<p>這裡有<strong>粗體</strong>  
和<em>斜體</em>。</p>
```

```
<!-- 換行（自關閉標籤） -->
```

```
<p>第一行<br>第二行</p>
```

<!-- 無序清單（圓點） -->

```
<ul>
  <li>蘋果</li>
  <li>香蕉</li>
</ul>
```

<!-- 有序清單（編號） -->

```
<ol>
  <li>第一步：開啟 Chrome</li>
  <li>第二步：前往高鐵網站</li>
</ol>
```

- ：unordered list（無序）
- ：ordered list（有序）
- ：list item（清單項目）


```
<!-- 超連結：點擊後前往指定網址 -->  
<a href="https://www.thsrc.com.tw">  
    高鐵官網  
</a>
```

```
<!-- 圖片：自關閉標籤 -->  

```

- `<a>` 的 `href` 指定目標網址
- `<img>` 的 `src` 指定圖片來源
- 兩個都是靠**屬性**提供資訊（下一節詳述）

```
<!-- div：區塊容器（佔滿整行） -->
```

```
<div>
```

```
  <p>段落一</p>
```

```
  <p>段落二</p>
```

```
</div>
```

```
<!-- span：行內容器（不換行） -->
```

```
<p>價格：<span>650</span> 元</p>
```

- <div> 和 本身**沒有視覺效果**
- 用來**分組**相關元素，方便套用 CSS 樣式或用程式定位
- <div> = **block**（佔一整行）； = **inline**（行內）

- 1 HTML 標籤的開始和結束有什麼區別？
- 2 為什麼 `<img>` 不需要結束標籤？
- 3 `<ul>` 和 `<ol>` 的差異？
- 4 `<div>` 和 `<span>` 各自的特性？何時用哪一個？
- 5 下面的 HTML 有語法錯誤，是什麼？

```
<p>這是<strong>粗體文字</p></strong>
```

- ❶ 結束標籤在 < 後面多一個 /，如 </p>。
- ❷ 是**自關閉標籤**，它只提供圖片來源，沒有「內容」需要包在裡面。
- ❸ 產生無序清單（圓點）； 產生有序清單（1, 2, 3...）。
- ❹ <div> 是 block 元素，佔滿一整行； 是 inline 元素，不換行。分組大區塊用 <div>，標記行內文字用 。
- ❺ 嵌套順序錯誤：應先關 再關 </p>。正確寫法：<p> 這是 粗體文字 </p>。

屬性 (Attribute)：給標籤附加資訊

屬性寫在**開始標籤**裡，格式 屬性名 =" 值"：

```
<input type="text" id="dateField"  
      name="date" value="2026/04/08">
```

- 一個標籤可以有**多個**屬性，用空格分隔
- 屬性提供標籤無法自行表達的額外資訊
- 不同標籤支援不同的屬性

常見屬性分類：

- **識別用**：id、class、name
- **內容用**：value、src、href
- **行為用**：type、disabled、checked

```
<button id="SubmitButton">開始查詢</button>
```

- id 在整個頁面中**必須唯一**（像身分證號碼）
- CSS 用 #id 定位；JavaScript 也用它找元素
- 高鐵頁面範例：id="BookingS1Form"、id="SubmitButton"

注意：高鐵頁面有個拼錯的 id：

```
<a id="cookieAccpetBtn">我同意</a>
```

Accpet 是錯的（應為 Accept），但程式必須**照原樣使用**。

```
<a class="btn-edit">修改</a>  
<div class="uk-modal uk-open">...</div>
```

- class 可以被**多個元素共用**（不像 id 必須唯一）
- 一個元素可以有**多個 class**，用空格分隔
- 上例的 <div> 同時有 uk-modal 和 uk-open 兩個 class
- CSS 用 .class 定位

```
<select name="selectStartStation">
  <option value="11">台南</option>
  <option value="7">台中</option>
</select>

<input type="text" name="toTrainIDInputField"
  value="0818">
```

- name：表單欄位的名稱，送出表單時用它當 key
- value：欄位的**目前值**
 - <input> 的 value = 輸入框裡的文字
 - <option> 的 value = 程式讀到的值（使用者看到的是標籤文字）

`<input>` 標籤靠 `type` 決定外觀和行為：

type 值	呈現
text	一般文字輸入框
password	密碼框（輸入顯示為 <code>*</code> ）
radio	單選圓鈕
checkbox	核取方塊
submit	送出按鈕

```
<input type="text" value=" 你的名字">
<input type="radio" name="seat" value="1">
<input type="submit" value=" 開始查詢">
```

HTML5 允許開發者用 data- 前綴自訂任意屬性：

```
<!-- thsr.R 用到的實例 -->
<input type="radio"
       data-target="search-by-trainNo">

<div data-user-id="12345"
     data-role="admin">...</div>
```

- data- 後面的名稱**自由定義**
- 不影響標籤原有功能，純粹用來**存放額外資訊**
- CSS 可用 [data-target='search-by-trainNo'] 選取
- JavaScript 可用 el.dataset.target 讀取

thsr.R 中的 input[data-target='search-by-trainNo'] 就是用屬性選擇器定位「車次查詢」radio。

- ❶ id 和 class 最大的差異？
- ❷ `<option value="11"> 台南 </option>` 中，程式讀到的值是 "11" 還是 " 台南"？
- ❸ 以下 `<input>` 目前顯示什麼文字？

```
<input type="text" name="trainNo" value="0818">
```

- ❹ 為什麼 name 屬性對表單很重要？

- ❶ id 在整頁**必須唯一**；class 可以被多個元素共用，且一個元素可有多個 class。
- ❷ 程式讀到 "11" (value 屬性)。使用者在下拉選單看到的是 " 台南" (標籤文字)。
- ❸ 顯示 0818 (value 屬性的值)。
- ❹ 表單送出時，name 當作 key、value 當作 value，組成 key-value pair 送給伺服器。沒有 name 的欄位不會被送出。

表單 (Form)：使用者輸入的容器

```
<form id="BookingS1Form" action="/submit"
      method="POST">
  <!-- 所有輸入欄位放在這裡 -->
  <input type="text" name="date">
  <button type="submit">送出</button>
</form>
```

- `<form>` 是所有輸入元素的**外層容器**
- `action`：表單送到哪個網址
- `method`：用 GET 或 POST 送出
- 高鐵訂票表單的 id 就是 BookingS1Form

```
<select name="selectStartStation">
  <option value="">請選擇</option>
  <option value="1">南港</option>
  <option value="2">台北</option>
  <option value="7">台中</option>
  <option value="11">台南</option>
  <option value="12">左營</option>
</select>
```

- `<select>` = 下拉選單容器
- `<option>` = 個別選項
- 使用者選「台南」→ 程式讀到 `value="11"`
- 這就是 `thsr.R` 中 `js_select(sess, "[name='selectStartStation']", "11")` 的由來

```
<!-- Radio：同 name 只能選一個 -->  
<input type="radio" name="seat" value="0">不限  
<input type="radio" name="seat" value="1">靠窗  
<input type="radio" name="seat" value="2">走道
```

```
<!-- Checkbox：可複選 -->  
<input type="checkbox" name="agree">
```

我已閱讀並同意

- Radio：同一個 name 的多個 radio 構成一組，**只能選一個**
- Checkbox：獨立的勾選框
- thsr.R 中 SEAT <- "1" 對應靠窗的 radio value

```
<!-- 方式一：<button> 標籤 -->
<button type="submit">開始查詢</button>

<!-- 方式二：<input type="submit"> -->
<input type="submit" id="SubmitButton"
      value=" 開始查詢">
```

- 兩種寫法功能相同，高鐵頁面用的是 `<input type="submit">`
- `value` 是按鈕上顯示的文字
- `id="SubmitButton"` → CSS Selector `#SubmitButton`


```
<form id="BookingS1Form">
  <select name="selectStartStation">
    <option value="11">台南</option>
    <option value="7">台中</option>
  </select>
  <input type="text" id="toTimeInputField"
    value="2026/04/08">
  <input type="radio" name="seat" value="1">
  <input type="text"
    name="homeCaptcha:securityCode">
  
  <input type="submit" id="SubmitButton"
    value=" 開始查詢">
</form>
```

這就是高鐵訂票頁面的簡化版 HTML。

- ❶ `<form>` 標籤的作用是什麼？
- ❷ 一組 radio button 靠什麼屬性綁在一起？為什麼只能選一個？
- ❸ 以下表單中有幾個可以讓使用者輸入資料的元素？

```
<form>
  <select name="s"><option>A</option></select>
  <input type="text" name="t">
  <input type="checkbox" name="c">
  
  <button type="submit">送出</button>
</form>
```

- ❶ `<form>` 是所有輸入元素的容器。使用者按送出時，瀏覽器把 `<form>` 內所有有 `name` 的欄位打包送給伺服器。
- ❷ 靠 `name` 屬性。同一個 `name` 的 `radio` 構成一組，瀏覽器強制只能選一個（選新的會自動取消舊的）。
- ❸ **3 個**：`<select>`、`<input type="text">`、`<input type="checkbox">`。`<img>` 是圖片不能輸入，`<button>` 是送出按鈕。

DOM：瀏覽器怎麼理解 HTML

瀏覽器收到 HTML 文字後，不是「直接顯示文字」：

- ❶ **解析**：逐行讀取標籤
- ❷ **建樹**：在記憶體中建立一棵**樹狀結構**
- ❸ **渲染**：根據樹畫出畫面

這棵樹叫做 **DOM (Document Object Model，文件物件模型)**。

- 每個 HTML 標籤 → 樹上的一個**節點 (node)**
- 嵌套關係 → 父子關係

```
<form id="BookingS1Form">
  <select name="selectStartStation">
    <option value="11">台南</option>
    <option value="7">台中</option>
  </select>
  <button id="SubmitButton">開始查詢</button>
</form>
```

```
form (id="BookingS1Form")
  select (name="selectStartStation")
    option (value="11") → "台南"
    option (value="7") → "台中"
  button (id="SubmitButton") → "開始查詢"
```

DOM 不是靜態快照 — 它是**活的**：

- 程式可以隨時**新增、刪除、修改**節點
- 瀏覽器畫面**立即同步更新**

例如：用程式把 `<select>` 的 `value` 改成 "11"

→ 畫面上的下拉選單立刻顯示「台南」

`thsr.R` 的核心就是：透過 `WebDriver` 遠端修改高鐵頁面的 DOM。

- ❶ 瀏覽器收到 HTML 後做了哪三件事？
- ❷ 以下 HTML 的 DOM 樹長什麼樣？

```
<div id="box"><input type="text"><button>送出</button></div>
```

- ❸ 為什麼說 DOM 是「活的」？這個特性對 thsr.R 為什麼重要？

❶ 解析 HTML → 建立 DOM 樹 → 渲染（畫出）畫面。

❷ 樹狀結構：

```
div (id="box")  
  input (type="text")  
  button → "送出"
```

❸ 程式修改 DOM 後瀏覽器立即更新畫面。thsr.R 靠遠端修改 DOM（改 value、觸發事件、點擊按鈕）來操作高鐵頁面。

CSS 基礎與 Selector

CSS (Cascading Style Sheets) = 控制網頁**外觀**的語言。
HTML 負責**結構** (有什麼元素)，CSS 負責**表現** (元素長什麼樣)。

```
<!-- HTML：結構 -->
```

```
<button id="SubmitButton">開始查詢</button>
```

```
/* CSS：外觀 */
```

```
#SubmitButton {  
  background-color: orange;  
  color: white;  
  font-size: 16px;  
}
```

高鐵頁面的橘色「開始查詢」按鈕就是這樣做出來的。

```
selector {  
  property: value;  
  property: value;  
}
```

- **selector**：指定要套用樣式的目標元素
- **property**：要改的外觀屬性（顏色、大小、邊距...）
- **value**：屬性的值

```
p      { color: red; }      /* 所有 <p> */  
#main  { width: 800px; }    /* id="main" */  
.active { font-weight: bold; } /* class="active" */
```

Selector 的語法後來被 JavaScript 和爬蟲工具廣泛採用。

CSS 如何套用到 HTML

三種方式（由近到遠）：

<!-- 1. 行內樣式（寫在標籤裡） -->

```
<p style="color: red;">紅色文字</p>
```

<!-- 2. 內嵌樣式（寫在 <head> 裡） -->

```
<head>
  <style>
    p { color: blue; }
  </style>
</head>
```

<!-- 3. 外部檔案（獨立 .css 檔） -->

```
<head>
  <link rel="stylesheet" href="style.css">
</head>
```

高鐵頁面用的是方式 2 和 3。對爬蟲來說，重要的不是樣式本身，而是 **selector 的語法**。

CSS Selector : tag、#id、.class

寫法	意思	範例
tag	所有該標籤	button → 所有 <button>
#xxx	id="xxx"	#SubmitButton
.xxx	class 含 xxx	.btn-edit

```
button      { cursor: pointer; }  
#SubmitButton { background: orange; }  
.btn-edit   { border: 1px solid blue; }
```

這是最基本的三種 selector，也是 thsr.R 中最常用的。

用方括號 `[]` 根據**屬性**篩選：

寫法	意思
<code>[name='xxx']</code>	name 屬性 等於 xxx
<code>[attr*='xxx']</code>	屬性值 包含 xxx
<code>[attr^='xxx']</code>	屬性值 開頭是 xxx
<code>[attr\$='xxx']</code>	屬性值 結尾是 xxx

```
[name='selectStartStation'] { ... }  
[href*='Cancel']           { ... }  
input[type='radio']        { ... }
```

最後一個是**組合**：標籤名 + 屬性，表示「`<input>` 且 type 是 radio」。

CSS Selector：層級選擇器

```
/* 後代選擇器：div 內所有的 p（不限層數） */  
div p { color: gray; }
```

```
/* 子選擇器：div 的直接子元素 p */  
div > p { color: blue; }
```

```
/* 多重選擇器：逗號分隔，同時選多個 */  
h1, h2, h3 { font-weight: bold; }
```

thsr.R 中的範例：

```
div.uk-modal.uk-open input.uk-modal-close
```

意思：在 class 同時含 uk-modal 和 uk-open 的 <div> 裡面，找 class 含 uk-modal-close 的 <input>。

- 1 CSS 的三個組成部分是什麼 (___ { ___: ___; })?
- 2 #main 和 .main 的差異?
- 3 [name='agree'] 會選到以下哪些元素?

```
<input type="checkbox" name="agree">  
<input type="text" name="agree">  
<input type="text" name="disagree">
```

- 4 input[type='radio'] 只會選到什麼元素?

- ❶ **selector** (選擇器)、**property** (屬姓名)、**value** (屬性值)。
- ❷ **#main** 找 `id="main"` (唯一)；**.main** 找 `class` 含 `main` (可多個)。
- ❸ 前兩個。它們的 `name` 都等於 `"agree"`。第三個是 `"disagree"`，不符合。
- ❹ 所有 `<input>` 標籤中 `type` 屬性為 `"radio"` 的元素。

Box Model：元素的空間結構

每個 HTML 元素都是一個**矩形盒子**，由內到外四層：

```
margin
border
padding

content
```

```
#SubmitButton {
  padding: 8px 16px; /* 內容與邊框間距 */
  border: 2px solid orange;
  margin: 10px;      /* 與其他元素間距 */
}
```

DevTools 的 Elements 面板右下角可視覺化看到 Box Model。

偽類 (Pseudo-class)

偽類描述元素的**狀態**，用 `:` 語法：

偽類	觸發時機
<code>:hover</code>	滑鼠移上去
<code>:focus</code>	欄位被聚焦（點進輸入框）
<code>:first-child</code>	父元素的第一個子元素
<code>:nth-child(n)</code>	父元素的第 n 個子元素
<code>:checked</code>	radio / checkbox 被勾選
<code>:disabled</code>	被停用的表單元素

```
input:focus { border-color: blue; }  
option:checked { background: yellow; }  
li:nth-child(2n) { background: #f0f0f0; }
```

爬蟲較少用偽類選取元素，但理解它有助於讀懂頁面的 CSS。

CSS Specificity (優先權)

當多條規則指向同一元素，哪條生效？**Specificity 高者贏。**

計算方式（由高到低）：

來源	權重	範例
行內 style=""	1000	style="color:red"
#id	100	#SubmitButton
.class / [attr] / :pseudo	10	.btn-edit, [name='x']
tag	1	button, input

```
button      { color: black; } /* 1 */
#SubmitButton { color: orange; } /* 100 → 贏 */
```

!important 蓋過一切（但不建議濫用）。了解 Specificity 可幫助你在 DevTools 中判斷哪條 CSS 規則真正生效。

```
wait_for(sess, "#BookingS1Form")
# → id="BookingS1Form"

wait_click(sess, "#SubmitButton")
# → id="SubmitButton"

js_select(sess,
  "[name='selectStartStation']", "11")
# → name="selectStartStation"

wait_click(sess, ".btn-edit")
# → class 含 "btn-edit"

wait_click(sess,
  "input[data-target='search-by-trainNo']")
# → <input> 且 data-target 屬性符合
```

- ❶ 寫出定位以下元素的 CSS Selector (用 id 和 name 各一種)：

```
<input id="mobilePhone" name="phone" type="text">
```

- ❷ [href*='Cancel'] 能定位到以下哪個元素？

```
<a href="/booking/CancelOrder">取消</a>
```

```
<a href="/booking/Confirm">確認</a>
```

- ❸ data-target 是什麼類型的 HTML 屬性？

- ❶ 用 `id: #mobilePhone` ; 用 `name: [name='phone']` 。
- ❷ 第一個 `<a>` , 因為它的 `href` **包含** `Cancel` 。
- ❸ `data-` 前綴是 HTML5 的**自訂資料屬性** (custom data attribute) , 開發者可自行定義, 用來存放額外資訊。CSS 和 JavaScript 都可以用它來選取元素。

JavaScript 基礎語法

JavaScript 是什麼

- 唯一能在瀏覽器裡**直接執行**的程式語言
- 能讀取和修改 DOM → **讓網頁動起來**

三者的分工：

技術	負責	比喻
HTML	結構	骨架
CSS	外觀	皮膚、衣服
JavaScript	行為	肌肉、神經

thsr.R 透過 WebDriver 把 JavaScript **注入**到 Chrome 裡執行。

- 1 瀏覽器 Console (F12 → Console 分頁)：直接輸入、立即執行
- 2 HTML 中的 `<script>` 標籤：

```
<script>  
  alert("Hello!");  
</script>
```

- 3 外部 .js 檔案：

```
<script src="app.js"></script>
```

- 4 thsr.R 的做法：透過 WebDriver 的 `/execute/sync` 端點，把 JS 字串送到 Chrome 執行

變數 (Variable)

變數 = 幫資料取一個名字，方便之後使用。

```
// var: 傳統寫法 (函式作用域)
```

```
var x = 10;
```

```
var name = "Alice";
```

```
// let: ES6 新語法 (區塊作用域，更嚴謹)
```

```
let count = 0;
```

```
// const: 常數 (設定後不可改)
```

```
const PI = 3.14159;
```

- var 和 let 的值可以改: `x = 20;`
- const 不可改: `PI = 3;` → 錯誤
- thsr.R 的 JavaScript 主要用 var

資料型別 (Data Type)

```
// 字串 (String) : 用引號包住
var s = "hello";
var s2 = 'world';           // 單引號也可以

// 數字 (Number)
var n = 42;
var pi = 3.14;

// 布林 (Boolean)
var ok = true;
var fail = false;

// null : 「刻意設為空」
var empty = null;

// undefined : 「尚未賦值」
var x;           // x 是 undefined
```

```
// 串接
"hello" + " " + "world"    // "hello world"

// 長度
"abc".length                // 3

// 去除前後空白
"  hi  ".trim()             // "hi"
// thsr.R 的 ocr_captcha 就用了 trimws

// 包含檢查
"hello".includes("ell")    // true

// 大小寫轉換
"Hello".toLowerCase()      // "hello"
"Hello".toUpperCase()      // "HELLO"
```

模板字串 (Template Literal)

ES6 引入用**反引號** ``` 包裹的字串，支援**插值**和**多行**：

```
// 插值：${expression}
var name = "Alice";
var msg = `Hello, ${name}!`;
// → "Hello, Alice!"
```

```
// 表達式
var price = 650;
var discount = 0.65;
`票價：${price * discount} 元`
// → " 票價：422.5 元"
```

```
// 多行字串 (不需要 \n)
var html = `
  <div>
    <p>Hello</p>
  </div>`;
```

thsr.R 使用傳統字串，但現代 JavaScript 廣泛使用模板字串。

比較運算子

```
// 嚴格相等 / 不相等 (推薦用這組)
1 === 1           // true
1 === "1"         // false (型別不同)
1 !== 2           // true

// 寬鬆相等 (會自動型別轉換，容易出 bug)
1 == "1"          // true ( 危險)

// 大小比較
5 > 3             // true
5 >= 5            // true
```

thsr.R 的 JavaScript 都用 ===，例如：

```
el.textContent.trim() === "\u53d6\u6d88\u8a02\u4f4d"
```

邏輯運算子與 if/else

```
// 邏輯運算子
true && false    // false (AND: 兩者皆真)
true || false    // true  (OR: 其一為真)
!true            // false (NOT: 反轉)

// 條件判斷
if (el !== null) {
  el.click();
} else {
  console.log(" 元素不存在");
}
```

thsr.R 中常見的模式：

```
if (a) { a.click(); return true; }
return false;
```


- ❶ `var`、`let`、`const` 的差異？
- ❷ `"hello" + " " + "world"` 的結果？
- ❸ `1 === "1"` 和 `1 == "1"` 各回傳什麼？為什麼？
- ❹ `" abc ".trim()` 的結果？
- ❺ 以下程式碼的輸出是什麼？

```
var x = 10;  
if (x > 5 && x < 20) {  
  console.log("yes");  
} else {  
  console.log("no");  
}
```

- ❶ `var` 函式作用域、`let` 區塊作用域、`const` 常數不可改。
- ❷ `"hello world"` (字串串接)。
- ❸ `1 === "1" → false` (嚴格相等，型別不同)。 `1 == "1" → true` (寬鬆相等，自動轉型)。
- ❹ `"abc"` (去除前後空白)。
- ❺ 輸出 `"yes"` (`10 > 5` 且 `10 < 20`，條件成立)。

函式 (Function)

函式 = 一段可重複使用的程式碼，有輸入（參數）和輸出（回傳值）。

```
// 宣告函式
function add(a, b) {
  return a + b;
}
```

```
// 呼叫函式
add(3, 5);    // 8
add(10, 20);  // 30
```

- function 關鍵字開頭
- return 把結果傳回去
- **沒有 return → 回傳 undefined** (thsr.R 中 wd_js 要記得用 return !)

匿名函式 (Anonymous Function)

函式可以沒有名字，直接當成**值**傳遞：

```
// 具名函式
function greet() { return "hi"; }

// 匿名函式 (存在變數裡)
var greet = function() { return "hi"; };

// 匿名函式當作參數傳給別人
setTimeout(function() {
  console.log("3 秒後執行");
}, 3000);
```

- 匿名函式常用於**回呼 (callback)**：「事情完成後，幫我執行這段程式」
- `thsr.R` 中大量使用，如 `addEventListener("load", function() {...})`

箭頭函式 (Arrow Function)

ES6 的簡潔函式語法：

```
// 傳統匿名函式  
nums.filter(function(n) { return n > 3; });
```

```
// 箭頭函式 (完全等價)  
nums.filter(n => n > 3);
```

```
// 多參數 / 多行  
var add = (a, b) => {  
  return a + b;  
};
```

```
// 單一表達式 (省略 {} 和 return)  
var add = (a, b) => a + b;
```

- thsr.R 的 JavaScript 用 function (相容舊版瀏覽器)
- 現代 JS 幾乎全面使用 =>
- **重要差異**：箭頭函式沒有自己的 this (繼承外層)

IIFE：立即執行函式

```
(function() {  
  var el = document.querySelector("#btn");  
  el.value = "hello";  
  el.dispatchEvent(new Event("change",  
    {bubbles: true}));  
})();
```

- **IIFE** = Immediately Invoked Function Expression
- 宣告一個匿名函式，**立刻執行它**
- 好處：var el 只存在函式內部，**不汙染全域命名空間**
- thsr.R 的 fill()、js_select()、click_link_text() 都用這個模式

閉包 (Closure)

函式可以「記住」它被**建立時**的變數環境：

```
function makeCounter() {  
  var count = 0;           // 外層變數  
  return function() {     // 內層函式  
    count += 1;           // 存取外層變數  
    return count;  
  };  
}
```

```
var counter = makeCounter();  
counter() // 1  
counter() // 2  
counter() // 3 — count 被「記住」了
```

- 內層函式 + 它記住的外層變數 = **閉包**
- 和 R 的 closure (`<<-`) 原理相同

閉包實例：IIFE 就是閉包

thsr.R 的 fill() 函式送出的 JavaScript：

```
(function() {  
  var el = document.querySelector("#btn");  
  el.value = "hello";  
  el.dispatchEvent(new Event("change",  
    {bubbles: true}));  
})();
```

IIFE 建立了一個閉包：

- var el 只存在於這個閉包內
- 外部無法存取或汙染 el
- 執行完畢後，閉包中的變數自動釋放

閉包的核心價值：資料封裝（data encapsulation）—— 不需要 class 也能隱藏內部狀態。


```
try {  
  // 可能出錯的程式碼  
  var el = document.querySelector("#xxx");  
  el.click(); // 如果 el 是 null → TypeError  
} catch (error) {  
  // 出錯時執行  
  console.log(" 錯誤：" + error.message);  
} finally {  
  // 不管有沒有出錯，一定執行  
  console.log(" 清理完成");  
}  
  
// 主動拋出錯誤  
function divide(a, b) {  
  if (b === 0) throw new Error(" 除以零");  
  return a / b;  
}
```

和 R 的 tryCatch 相似。thsr.R 在 R 層用 tryCatch 處理錯誤，JS 層靠 if (el) 防禦。

- ❶ 沒有 `return` 的函式回傳什麼？這對 `wd_js()` 有什麼影響？
- ❷ 什麼是匿名函式？它通常用在什麼場合？
- ❸ 將 `nums.filter(function(n) { return n > 3; })` 改寫為箭頭函式。
- ❹ 以下 IIFE 執行後，全域變數裡有 `e1` 嗎？為什麼？

```
(function() {  
  var e1 = document.querySelector("p");  
  e1.click();  
})();
```

- ❺ 以下 `makeGreeter` 是閉包嗎？呼叫 `hi("Alice")` 的結果？

```
function makeGreeter(greeting) {  
  return function(name) {  
    return greeting + ", " + name;  
  };  
}  
  
var hi = makeGreeter("Hello");
```

- ❶ 回傳 `undefined`。wd_js 會收到 `NULL`。所有要回傳結果的 JS 必須寫 `return`。
- ❷ 沒有名字的函式。常用作 `callback`（事件監聽、`.find()`、`setTimeout`）。
- ❸ `nums.filter(n => n > 3)`
- ❹ **沒有**。var el 在 IIFE 的函式作用域內，執行完即消失。這就是閉包的資料封裝。
- ❺ **是閉包**。內層函式記住了外層的 `greeting` 變數。`hi("Alice")` → `"Hello, Alice"`。

陣列 (Array)

```
// 建立陣列
var fruits = ["apple", "banana", "cherry"];

// 存取元素 (從 0 開始)
fruits[0]      // "apple"
fruits[2]      // "cherry"
fruits.length  // 3

// 修改
fruits[1] = "blueberry";

// 新增到尾端
fruits.push("date");
```

陣列方法：forEach、map、filter

```
var nums = [1, 2, 3, 4, 5];

// forEach：對每個元素執行函式
nums.forEach(function(n) {
  console.log(n);
});

// map：轉換每個元素，回傳新陣列
nums.map(function(n) { return n * 2; });
// → [2, 4, 6, 8, 10]

// filter：篩選符合條件的元素
nums.filter(function(n) { return n > 3; });
// → [4, 5]
```

Array.from 與.find

```
// querySelectorAll 回傳 NodeList (不是 Array)
var links = document.querySelectorAll("a");
// links.find(...) → 錯誤! NodeList 沒有 .find

// Array.from: 轉成真正的 Array
var arr = Array.from(links);
// arr.find(...) → 可以了!

// .find: 回傳第一個符合條件的元素
var target = arr.find(function(el) {
  return el.textContent.trim() === " 取消訂位";
});
```

thsr.R 的 `click_link_text()` 就是這個模式。

物件 (Object)

物件 = **key-value** 組合，類似 R 的 list：

```
var rect = {  
  x: 100,  
  y: 200,  
  width: 50,  
  height: 30  
};  
  
// 存取屬性  
rect.x           // 100  
rect["width"]    // 50  
  
// 回傳物件  
function getInfo() {  
  return { name: "Alice", age: 25 };  
}
```

WebDriver 的 JSON 回應就是物件；getBoundingClientRect() 回傳的也是物件。

JSON : JavaScript 物件表示法

JSON = JavaScript Object Notation , 資料交換的通用格式。

```
// JS 物件 → JSON 字串
var obj = { name: "THSR", year: 2007 };
JSON.stringify(obj)
// → '{"name":"THSR","year":2007}'

// JSON 字串 → JS 物件
var text = '{"x":100,"y":200}';
var rect = JSON.parse(text);
rect.x    // 100
```

WebDriver 協定全部使用 JSON 通訊：

- R 的 toJSON() = JavaScript 的 JSON.stringify()
- R 的 fromJSON() = JavaScript 的 JSON.parse()
- thsr.R 的 wd_cmd 用 fromJSON(rawToChar(res\$content)) 解析 ChromeDriver 的 JSON 回應

- 1 ["a", "b", "c"][1] 的值是什麼？
- 2 Array.from 的用途？為什麼 querySelectorAll 的結果需要它？
- 3 .find() 和 .filter() 的差異？
- 4 以下程式碼的結果是什麼？

```
var obj = { name: "THSR", year: 2007 };  
obj.name + " " + obj.year
```

- ❶ "b" (陣列索引從 0 開始)。
- ❷ 把「長得像陣列但不是陣列」的東西 (如 `NodeList`) 轉成真正的 `Array`。 `NodeList` 沒有 `.find()`、`.map()` 等方法，轉換後才能使用。
- ❸ `.find()` 回傳**第一個**符合條件的元素；`.filter()` 回傳**所有**符合的元素 (陣列)。
- ❹ "THSR 2007" (字串串接，數字自動轉型)。

JavaScript DOM 操作

document 是 DOM 樹的**根節點**，所有 DOM 操作從它開始：

```
// 用 CSS Selector 找第一個符合的元素
document.querySelector("#SubmitButton")

// 找所有符合的元素 (回傳 NodeList)
document.querySelectorAll("a")

// 找不到 → 回傳 null
document.querySelector("# 不存在的 id")
// → null
```

thsr.R 的 wd_js() 送到瀏覽器執行的 JavaScript 幾乎都以 document.querySelector 開頭。

querySelector 與 null 檢查

```
// 找到元素：回傳該元素
var el = document.querySelector("#SubmitButton");
// el 是一個 DOM 元素物件

// 找不到元素：回傳 null
var el = document.querySelector("#xxx");
// el 是 null

// 安全檢查：先確認不是 null 再操作
if (el !== null) {
    el.click();
}

// 簡寫 (null 在 if 中視為 false)
if (el) {
    el.click();
}
```

thsr.R 中 `wait_for()` 的核心就是反覆檢查 `querySelector(...) !== null`。

```
var el = document.querySelector(
    "[name='selectStartStation']");

// 讀取表單欄位的值
el.value           // "11"

// 讀取元素內的文字
el.textContent     // " 台南台中..."

// 讀取標籤名稱
el.tagName         // "SELECT"

// 讀取任意屬性
el.getAttribute("name") // "selectStartStation"
el.id              // "" (沒有 id)

// 檢查圖片是否載入完成
el.complete        // true 或 false (僅 <img>)
```

```
var el = document.querySelector("#toTimeInputField");
```

```
// 設定值
```

```
el.value = "2026/04/08";
```

```
// 設定任意屬性
```

```
el.setAttribute("disabled", "true");
```

```
// 聚焦（模擬使用者點進輸入框）
```

```
el.focus();
```

```
// 模擬點擊
```

```
el.click();
```

thsr.R 的 fill() = 設定 value + focus() thsr.R 的 wait_click() = click()

getBoundingClientRect()

取得元素在畫面上的**位置和大小**：

```
var el = document.querySelector("#captchaImg");
var rect = el.getBoundingClientRect();
// rect = {
//   x: 950, y: 600,
//   width: 100, height: 40,
//   top: 600, left: 950,
//   bottom: 640, right: 1050
// }
```

- 座標以**瀏覽器可視區域左上角**為原點
- 回傳值是 CSS 像素（非螢幕實際像素）
- thsr.R 的 `element_screenshot()` 用它定位驗證碼圖片，再從全頁截圖中裁切出來

- ❶ `document.querySelector` 找不到元素時回傳什麼？
- ❷ `.value` 和 `.textContent` 的差異？
- ❸ 以下程式碼做了什麼？

```
var el = document.querySelector("#dateField");  
el.value = "2026/04/08";  
el.focus();
```

- ❹ `getBoundingClientRect()` 回傳的物件包含哪些資訊？

- ❶ 回傳 null (不是 undefined, 不是 error)。
- ❷ .value 是表單元素 (<input>、<select>) 的**目前值**。 .textContent 是元素**內部的文字內容** (包括子元素的文字)。
- ❸ 找到 id="dateField" 的元素 → 將其值設為 "2026/04/08" → 模擬使用者「點進」該輸入框 (聚焦)。
- ❹ 元素的位置 (x/y/top/left/bottom/right) 和大小 (width/height), 單位為 CSS 像素。

JavaScript 事件機制

事件（Event）是什麼

使用者在網頁上的每個動作，瀏覽器都會產生**事件**：

使用者動作	事件名稱
點擊元素	<code>click</code>
在輸入框打字	<code>input</code>
改變下拉選單選項	<code>change</code>
聚焦到欄位	<code>focus</code>
離開欄位	<code>blur</code>
圖片載入完成	<code>load</code>
圖片載入失敗	<code>error</code>

沒有事件機制，網頁就是靜態文件。

addEventListener：監聽事件

```
// 語法
element.addEventListener(" 事件名", handler);

// 範例：監聽下拉選單的 change 事件
var sel = document.querySelector(
    "[name='selectStartStation']");
sel.addEventListener("change", function() {
    console.log(" 出發站被改了！");
    console.log(" 新值：" + sel.value);
});
```

- handler 是一個函式（通常是匿名函式）
- 事件發生時，瀏覽器**自動呼叫** handler
- 一個元素可以註冊**多個**監聽器

事件冒泡 (Event Bubbling)

事件會從觸發元素**向上傳遞**到父元素、祖父元素...直到 document :

option → select → form → body → document
(觸發) (收到) (收到) (收到) (收到)

```
// 在父層 form 監聽，也能收到子元素的事件
document.querySelector("form")
  .addEventListener("change", function() {
    console.log(" 表單裡有東西被改了");
  });
```

- 這叫做**事件冒泡 (bubbling)**
- thsr.R 中 {bubbles: true} 就是啟用這個機制

dispatchEvent：手動觸發事件

核心問題：程式改 value **不會**自動觸發事件！

```
var el = document.querySelector(
  "[name='selectStartStation']");

// 只改值：頁面連動欄位不會更新
el.value = "11";

// 改值 + 手動觸發事件
el.value = "11";
el.dispatchEvent(
  new Event("change", {bubbles: true}));
```

- new Event("change") 建立一個 change 事件物件
- {bubbles: true} 讓事件向上冒泡
- dispatchEvent() 把事件「射出去」
- **這是 thsr.R 中 js_select() 和 fill() 的核心邏輯**

- ❶ 使用者在輸入框打字，瀏覽器產生什麼事件？
- ❷ 為什麼「程式改 value」不會觸發 change 事件？
- ❸ {bubbles: true} 的作用？不加會怎樣？
- ❹ 以下程式碼少了什麼？

```
var el = document.querySelector("#toTimeInputField");  
el.value = "2026/04/08";
```


- ❶ input 事件（每按一個鍵就觸發一次）。離開輸入框時還會觸發 change。
- ❷ 事件機制只對**使用者操作**（滑鼠、鍵盤）觸發。程式直接改 value 是 DOM 層級操作，瀏覽器不視為「使用者動作」。
- ❸ 讓事件向上冒泡傳遞給父元素。不加的話，只有被觸發的元素本身收得到事件，父層的監聽器不會被通知。
- ❹ 少了 `dispatchEvent`：`el.dispatchEvent(new Event("change", {bubbles: true}));`

Promise = 代表一個**未來才會完成**的操作。

```
var p = new Promise(function(resolve, reject) {  
  // 非同步操作 (例如等圖片載入)  
  var img = document.querySelector("img");  
  img.addEventListener("load", function() {  
    resolve(true);    // 成功  
  });  
  img.addEventListener("error", function() {  
    reject(" 載入失敗"); // 失敗  
  });  
});
```

- `resolve(value)`：操作成功，傳回結果
- `reject(reason)`：操作失敗，傳回原因
- `thsr.R` 的 `element_screenshot()` 用 `Promise` 等待驗證碼圖片載入

setTimeout

```
// 延遲執行
setTimeout(function() {
  console.log("3 秒後印出");
}, 3000);    // 毫秒

// 在 Promise 中當超時保底
new Promise(function(resolve) {
  el.addEventListener("load", function() {
    resolve(true);
  });
  setTimeout(function() {
    resolve(false); // 5 秒超時 → 放棄等待
  }, 5000);
});
```

- `setTimeout(fn, ms)` 在 `ms` 毫秒後執行 `fn`
- `thsr.R` 中用作 `Promise` 的超時機制

- ❶ Promise 的三種狀態是什麼？
- ❷ resolve 和 reject 分別代表什麼？
- ❸ 以下 Promise 最終的結果是 true 還是 false？

```
new Promise(function(resolve) {  
  setTimeout(function() {  
    resolve(true);  
  }, 1000);  
  setTimeout(function() {  
    resolve(false);  
  }, 2000);  
});
```

- ❶ **pending** (等待中)、**fulfilled** (已成功)、**rejected** (已失敗)。
- ❷ **resolve** = 告知「操作成功」並傳回結果。**reject** = 告知「操作失敗」並傳回原因。
- ❸ **true**。Promise 只會被 **resolve/reject 一次**。第一個 **setTimeout (1 秒)** 先觸發，**resolve(true)** 生效。第二個 **setTimeout (2 秒)** 的 **resolve(false)** 被忽略。

Promise 鏈：.then() 與.catch()

Promise 可以**串接**，形成處理鏈：

```
fetch("https://api.example.com/data")
  .then(function(response) {
    return response.json();    // 也回傳 Promise
  })
  .then(function(data) {
    console.log(data.name);    // 處理資料
  })
  .catch(function(error) {
    console.log(" 失敗：" + error.message);
  });
```

- .then(fn) — Promise 成功後執行
- .catch(fn) — 鏈中任何一步失敗都會跳到這裡
- 每個 .then 回傳新的 Promise → 可無限串接

ES2017 引入 async/await，讓非同步程式碼**讀起來像同步**：

```
// Promise 鏈版本
fetch(url)
  .then(function(r) { return r.json(); })
  .then(function(data) { console.log(data); });

// async/await 版本（完全等價）
async function getData(url) {
  var response = await fetch(url);
  var data = await response.json();
  console.log(data);
}
```

- async 標記函式為「非同步函式」
- await 暫停執行，等 Promise 解析完再繼續
- 錯誤用 try/catch 捕獲（和同步程式一樣）

element_screenshot() 等待驗證碼圖片載入：

```
// WebDriver /execute/sync 自動等待 Promise
return new Promise(function(resolve) {
  el.addEventListener("load",
    function() { resolve(true); });
  el.addEventListener("error",
    function() { resolve(false); });
  setTimeout(function() {
    resolve(false); // 5 秒超時
  }, 5000);
});
```

三個可能的 resolve 路徑（先觸發者贏）：

- ❶ 圖片載入成功 → resolve(true)
- ❷ 圖片載入失敗 → resolve(false)
- ❸ 5 秒超時 → resolve(false)

WebDriver 的 /execute/sync 端點**自動等待** Promise resolve 後才回傳結果給 R。

JS 用 `/pattern/flags` 語法（不需要像 R 雙重跳脫 `\\\\`）：

```
// 建立正規表達式
var re = /^[a-zA-Z0-9]+$/;

// .test(): 回傳 true/false
re.test("abc123")    // true
re.test("abc 123")   // false

// .match(): 回傳匹配結果
"2026-04-13".match(/(\d{4})-(\d{2})-(\d{2})/)
// → ["2026-04-13", "2026", "04", "13"]

// .replace(): 替換
"abc 123".replace(/\d+/, "***")
// → "abc ***"
```

- R 的 `grepl` JS 的 `.test()`
- R 的 `regmatches` JS 的 `.match()`
- R 的 `gsub` JS 的 `.replace(/pattern/g, ...)`

JavaScript 的 this 指向**呼叫方法的物件**：

```
var el = document.querySelector("#btn");
```

```
// el.click() — this 指向 el  
el.click();
```

```
// el.addEventListener 中的 this  
el.addEventListener("load", function() {  
    console.log(this.tagName); // "IMG"  
    // this === el  
});
```

```
// 箭頭函式中 this 繼承外層  
el.addEventListener("load", () => {  
    console.log(this); // 不是 el!  
});
```

- el.method() 模式：this = . 前面的物件
- thsr.R 中 el.focus()、el.click()、el.dispatchEvent(...) 都依賴這個機制

- 1 JSON.stringify({a: 1, b: "hello"}) 的結果？
- 2 Promise 被 resolve 兩次 (如 resolve(1); resolve(2))，最終值是什麼？
- 3 把以下 Promise 鏈改寫為 async/await：

```
fetch(url)
  .then(function(r) { return r.json(); })
  .then(function(d) { return d.name; });
```

- 4 JavaScript 的 /\d+/ 相當於 R 的什麼 pattern？
- 5 "abc123".match(/[a-z]+)/ 的結果？

- ❶ '{"a":1,"b":"hello"}' (JSON 字串)。
- ❷ 1。Promise 只能 resolve **一次**，第二次被忽略。
- ❸

```
async function getName(url) {  
  var r = await fetch(url);  
  var d = await r.json();  
  return d.name;  
}
```

- ❹ "\\d+" (R 中正規表達式的反斜線需要雙重跳脫)。
- ❺ ["abc"] (匹配第一個連續小寫字母序列)。

DevTools 實戰

- 1 Chrome → <https://irs.thsrc.com.tw/IMINT/?locale=tw>
- 2 按 **F12** (Win/Linux) 或 **Cmd+Option+I** (Mac)
- 3 頂端分頁：**Elements** (DOM)、**Console** (執行 JS)

Elements 面板：

- 右鍵點擊任何頁面元素 → 「**檢查**」 → 跳到 DOM 節點
- 查看 id、name、value、class 等屬性
- **Ctrl+F** 搜尋 DOM 文字

在 Console 輸入 JavaScript，按 Enter 立即執行：

```
// 確認按鈕存在
document.querySelector("#SubmitButton") !== null
// → true

// 讀取出發站值（未選時是空字串）
document.querySelector(
  "[name='selectStartStation']").value
// → ""

// 設值 + 觸發事件
var el = document.querySelector(
  "[name='selectStartStation']");
el.value = "11";
el.dispatchEvent(
  new Event("change", {bubbles: true}));
```

- ① 如何用 DevTools 找出「出發站」下拉選單的 name 屬性？
- ② 在 Console 輸入 `document.querySelector("#BookingS1Form") !== null`，預期結果？
- ③ 如何在 Elements 面板找到驗證碼圖片的 id？

- ❶ 右鍵點擊「出發站」下拉選單 → 「檢查」→ Elements 面板顯示 `<select name="selectStartStation" ...>`。
- ❷ `true`（訂票表單已載入時）。若頁面未載入完成，可能是 `false`。
- ❸ Elements 面板按 **Ctrl+F** 搜尋 `passCode`，找到 `<img id="BookingS1Form_homeCaptcha_passCode" ...>`。

HTTP 與 WebDriver

你的電腦

高鐵伺服器

```
GET /IMINT/?locale=tw
```

```
HTTP/1.1 200 OK
```

```
<html>...</html>
```

- **GET**：「給我這個頁面」（資料在 URL 上）
- **POST**：「送這些資料給你」（資料在 body 裡）

```
library(httr)

# GET 請求：取得網頁
res <- GET("https://irs.thsrc.com.tw/...")
html <- rawToChar(res$content)

# POST 請求：送出表單
res <- POST("https://example.com/login",
  body = list(user = "alice", pwd = "1234"),
  encode = "form")
```

- `res$content`：回應的原始位元組 (raw vector)
- `rawToChar()`：轉成字串

用 `rvest` 等靜態爬蟲拿 **HTML 可行**，但互動做不到：

- ❶ **下拉選單連動**：靠 JS `change` 事件更新，HTTP 模擬不了
- ❷ **驗證碼**：需瀏覽器渲染後才能截取圖片
- ❸ **多步驟有狀態**：每步依賴前步結果（session cookie 等）

解法：啟動一個**真實的 Chrome**，讓 R 遠端控制它。

你的 R 程式

HTTP 請求 (JSON 格式)

"請打開這個網址"

"請執行這段 JavaScript"

"請給我截圖"

ChromeDriver (本機, port 4444)

Chrome DevTools Protocol

Chrome 瀏覽器 (顯示視窗或背景執行)

R 只需要會發 HTTP — 和 `httr::GET()` 完全一樣。

每個受控瀏覽器 = 一個 **Session**，有唯一 Session ID：

POST /session → 開啟 Chrome
 ← { sessionId: "7db2..." }

POST /session/7db2.../url → 前往網址

POST /session/7db2.../execute/sync
 → 執行 JavaScript

GET /session/7db2.../screenshot
 → 取全頁截圖 (base64)

DELETE /session/7db2... → 關閉視窗

- ① HTTP 的 GET 和 POST 主要差異？
- ② 為什麼不能用 `rvest` 完成高鐵訂票？
- ③ POST `/session` 回傳的 `sessionId` 有什麼用途？

- ❶ GET 資料附在 URL 後面；POST 放在請求 body 裡（適合傳大量或敏感資料）。
- ❷ 訂票需要 JS 互動（下拉選單連動、驗證碼渲染），純 HTTP 請求做不到。
- ❸ 後續所有指令 URL 都要帶此 ID，以識別控制的是哪個瀏覽器實例。

thsr.R 逐段解析：環境設定

```
local({
  pkgs <- c("httr", "jsonlite",
            "magick", "base64enc")
  missing <- pkgs[!vapply(pkgs,
    requireNamespace, logical(1),
    quietly = TRUE)]
  if (length(missing)) {
    message(sprintf(" 安裝缺少的套件：%s",
      paste(missing, collapse = ", ")))
    install.packages(missing, quiet = TRUE)
    still <- missing[!vapply(missing,
      requireNamespace, logical(1),
      quietly = TRUE)]
    if (length(still))
      stop(sprintf(" 以下套件安裝失敗：%s",
        paste(still, collapse = ", ")))
  }
})
```

local() 的意義與載入套件

local({...}) 的作用：

- 在獨立環境執行，pkgs、missing 等變數用完即丟
- 不會汙染全域命名空間

```
library(httr); library(jsonlite)
library(magick); library(base64enc)
```

套件	用途
httr	發 HTTP 請求（與 ChromeDriver 通訊）
jsonlite	R 物件 JSON（WebDriver 用 JSON）
magick	圖片處理（截圖裁切、顯示驗證碼）
base64enc	Base64 解碼（截圖以 base64 傳輸）

```
setwd(dirname(  
  rstudioapi::getSourceEditorContext()$path))
```

- `rstudioapi::getSourceEditorContext()$path`：取得**目前在 RStudio 編輯的檔案**之完整路徑
- `dirname()`：取出目錄部分（去掉檔名）
- `setwd()`：設定工作目錄 = 程式所在的資料夾

效果：之後所有相對路徑（如 `"captcha.png"`、`"./chromedriver"`）都相對於程式所在處。

- ❶ `local({...})` 和直接寫在全域有什麼差異？
- ❷ `vapply(pkgs, requireNamespace, logical(1))` 回傳什麼型別的向量？
- ❸ 為什麼 `install.packages` 之後還要再 `vapply` 一次？
- ❹ 若不在 RStudio 中執行，`rstudioapi::getSourceEditorContext()` 會怎樣？

- ❶ `local()` 內的變數不留在全域環境，避免命名汙染。直接寫在全域的變數會一直存在。
- ❷ **logical 向量**，每個元素 TRUE（已安裝）或 FALSE（未安裝）。`logical(1)` 是指定回傳型別的模板。
- ❸ 確認安裝是否成功。某些套件可能因缺少系統相依（如 `libmagick`）而安裝失敗。
- ❹ 會出錯（`rstudioapi` 不可用）。此時需手動呼叫 `setwd()`。

thsr.R 逐段解析：chromedriver 自動下載

detect_chrome_version(): 為什麼需要?

- chromedriver 必須與 Chrome 的**主版本號 (milestone)** 一致
- 例: Chrome 136.x → 需要 chromedriver 136.x
- 程式先偵測本機 Chrome 版本, 再下載對應 chromedriver

```
detect_chrome_version <- function() {  
  sysname <- Sys.info()[["sysname"]]  
  ver <- NULL  
  if (sysname == "Windows") {  
    # 從 Registry BLBeacon\version 讀取  
    for (hive in c("HKLM", "HKCU")) {  
      out <- tryCatch(system2("reg",  
        args = c("query", paste0(hive,  
          "\\SOFTWARE\\Google\\Chrome",  
          "\\BLBeacon"), "/v", "version"),  
        stdout = TRUE, stderr = NULL),  
        error = function(e) NULL)  
      ...  
    }
```

```
} else if (sysname == "Darwin") {  
  out <- tryCatch(system2(paste0(  
    "/Applications/Google Chrome.app/",  
    "Contents/MacOS/Google Chrome"),  
    args = "--version", stdout = TRUE,  
    stderr = NULL), error = function(e) NULL)  
} else {  
  for (cmd in c("google-chrome", "google-chrome-stable",  
    "chromium-browser", "chromium")) {  
    out <- tryCatch(system2(cmd,  
      args = "--version", stdout = TRUE,  
      stderr = NULL), error = function(e) NULL)  
    if (!is.null(out)) break  
  }  
}
```

- 統一用 `regexpr()` 提取版本號；偵測失敗回傳 `NULL` → 退回最新 Stable

ensure_chromedriver(): 函式簽名

```
ensure_chromedriver <- function(  
  path = "./chromedriver") {  
  exe <- if (.Platform$OS.type == "windows" &&  
    !grepl("\\.exe$", path))  
    paste0(path, ".exe") else path  
  if (file.exists(exe)) {  
    message("chromedriver 已存在")  
    return(invisible(exe))  
  }  
  message("chromedriver 不存在，自動下載中...")  
}
```

- Windows 自動補 .exe 副檔名
- 檔案已存在 → return(invisible(exe)) 直接回傳
- invisible(): 回傳值但不自動印在 Console

```
sysname <- Sys.info()[["sysname"]]
machine <- Sys.info()[["machine"]]
platform <- if (sysname == "Windows") {
  if (.Machine$sizeof.pointer == 8L)
    "win64" else "win32"
} else if (sysname == "Darwin") {
  if (grepl("arm|aarch", machine,
           ignore.case = TRUE))
    "mac-arm64" else "mac-x64"
} else { "linux64" }
```

- Sys.info()["sysname"] : "Windows" / "Darwin" / "Linux"
- .Machine\$sizeof.pointer == 8L : 64 位元系統
- macOS 區分 Intel (mac-x64) 和 Apple Silicon (mac-arm64)

ensure_chromedriver(): 偵測 Chrome 並查詢對應版本

```
chrome_ver <- detect_chrome_version()
if (!is.null(chrome_ver)) {
  major <- sub("\\..*", "", chrome_ver)
  api_url <- paste0(
    "https://googlechromelabs.github.io/",
    "chrome-for-testing/",
    "latest-versions-per-milestone-",
    "with-downloads.json")
  info <- fromJSON(content(
    GET(api_url), "text", encoding = "UTF-8"))
  milestone <- info$milestones[[major]]
  urls <- milestone$downloads$chromedriver
  dl <- urls[urls$platform == platform, "url"]
}
```

- `sub("\\..*", "", "136.0.7103.92")` → "136" (主版本號)
- **per-milestone API**: 查詢該 milestone 最新的 chromedriver

```
if (length(dl) == 0L) {  
  api_url <- paste0(  
    "https://googlechromelabs.github.io/",  
    "chrome-for-testing/",  
    "last-known-good-versions-"  
    "with-downloads.json")  
  info <- fromJSON(content(  
    GET(api_url), "text", encoding = "UTF-8"))  
  stable <- info$channels$Stable  
  urls <- stable$downloads$chromedriver  
  dl <- urls[urls$platform == platform, "url"]  
}
```

- 偵測失敗或主版本無對應 → 退回最新 **Stable** 版
- 兩層 API : **per-milestone** (精確) → **last-known-good** (退回)

ensure_chromedriver(): 解壓縮與安裝

```
zip_file <- tempfile(fileext = ".zip")
download.file(dl, zip_file,
              mode = "wb", quiet = TRUE)
tmp_dir <- tempfile()
unzip(zip_file, exdir = tmp_dir)
found <- list.files(tmp_dir,
                    pattern = "^chromedriver(\\.exe)?$",
                    recursive = TRUE, full.names = TRUE)
if (!length(found)) stop(" 找不到 chromedriver")
file.copy(found[1], exe, overwrite = TRUE)
if (.Platform$OS.type != "windows")
  Sys.chmod(exe, "755")
unlink(c(zip_file, tmp_dir), recursive = TRUE)
invisible(exe)
}
```

呼叫 `ensure_chromedriver()`

```
CHROMEDRIVER_PATH <- ensure_chromedriver()
```

- 首次執行：偵測 Chrome 版本 → 比對平台 → 下載 → 解壓 → 安裝
- 之後執行：檔案已存在，直接回傳路徑
- 回傳值存入 `CHROMEDRIVER_PATH` 供 `start_driver()` 使用

- ❶ `detect_chrome_version()` 為什麼要嘗試多種偵測方式 (Registry、PowerShell、`--version`)？
- ❷ `sub("\\..*", "", "136.0.7103.92")` 回傳什麼？為什麼用主版本號而非完整版本號查詢？
- ❸ `download.file(..., mode = "wb")` 中 `"wb"` 的 `b` 是什麼意思？
- ❹ 為什麼需要「退回機制」(fallback)？什麼情況下 per-milestone API 會查不到？

- ❶ 不同系統和安裝方式，Chrome 存放位置不同。Registry 是最標準的 Windows 做法，但企業環境可能只有 `chrome.exe` 路徑可用。Linux 則有多個可能的指令名稱。
- ❷ 回傳 "136"。chromedriver 以**主版本號 (milestone)** 為單位發布——只要主版本號相同即可相容，不需精確到 patch 號。
- ❸ b = binary。ZIP 是二進位檔案，Windows 上不加 b 可能被當文字處理而損壞。
- ❹ Chrome Canary / Dev 版本的主版本號可能尚未有對應 chromedriver；或偵測到的 Chrome 已過時。此時退回最新 Stable 版是安全的預設。

thsr.R 逐段解析：設定值

```
PID      <- "E111919345"  # 身分證號
MOBILE   <- "0912345678"  # 手機
MEMBER_PID <- "E111919345"  # TGo 會員號碼
SEAT     <- "1"           # 0= 不限 1= 靠窗 2= 走道
```

```
NUM_TRIALS  <- 10    # 驗證碼最多嘗試次數
WAIT_TIMEOUT <- 20    # 等待元素出現上限 (秒)
POLL_FREQ   <- 0.4    # 輪詢間隔 (秒)
```

```
train_id  <- "0818"  # 車次
start_sta <- "11"     # 出發站：台南
dest_sta  <- "7"      # 到達站：台中
```

車站代碼從 DevTools 查 <option value> 取得 (見附錄)。

```
THSR_BOOK_URL <- paste0(  
  "https://irs.thsrc.com.tw/IMINT/",  
  "?locale=tw")
```

```
THSR_CANCEL_URL <- paste0(  
  "https://irs.thsrc.com.tw/IMINT/",  
  "?wicket:bookmarkablePage=",  
  ":tw.com.mitac.webapp.thsr.",  
  "viewer.History")
```

- THSR_BOOK_URL：訂票頁面
- THSR_CANCEL_URL：取消訂位頁面
- 用 `paste0()` 拆成多行避免程式碼過長

thsr.R 逐段解析：WebDriver 通訊

`.wd_base()`：組出 Session URL 前綴

```
.wd_base <- function(sess) {  
  sprintf("http://localhost:%d/session/%s",  
    sess$port, sess$id)  
}
```

- 所有 WebDriver 操作 URL 都以此為前綴
- `sess`：R list，含 `id`（Session ID 字串）和 `port`（整數）
- `sprintf`：%d 代入整數，%s 代入字串
- 範例輸出："http://localhost:4444/session/7db27c7e..."

wd_cmd(): 通用 HTTP 命令

```
wd_cmd <- function(sess, method, path,  
                    body = NULL) {  
  url <- paste0(.wd_base(sess), path)  
  res <- if (method == "GET") GET(url)  
    else if (method == "POST")  
      POST(url, body = body, encode = "json")  
    else if (method == "DELETE") DELETE(url)  
  parsed <- fromJSON(rawToChar(res$content))  
  parsed$value  
}
```

- ChromeDriver 所有回應格式: { "value": < 實際資料 > }
- rawToChar(res\$content): raw bytes → 字串
- fromJSON(...): JSON 字串 → R list
- \$value: 取出實際資料

- ❶ `.wd_base(list(port = 4444, id = "abc"))` 的回傳值？
- ❷ `wd_cmd` 的 `body` 預設為 `NULL`，為什麼？
- ❸ `fromJSON(rawToChar(res$content))$value` 這一行做了哪三件事？

- ❶ `"http://localhost:4444/session/abc"`
- ❷ GET 和 DELETE 不需要 body (只有路徑)，只有 POST 才有 body 資料。
- ❸ 三件事：
 - `rawToChar()`：位元組轉字串
 - `fromJSON()`：JSON 解析成 R list
 - `$value`：取出 ChromeDriver 回應中的實際資料

start_driver(): 清除殘存行程

```
start_driver <- function(headless = TRUE,  
  port = 4444L,  
  driver_path = CHROMEDRIVER_PATH) {  
  if (.Platform$OS.type == "windows") {  
    system2("taskkill",  
      args = c("/F", "/IM",  
        "chromedriver.exe"),  
      wait = TRUE,  
      stdout = NULL, stderr = NULL)  
  } else {  
    system2("pkill", args = "chromedriver",  
      wait = TRUE,  
      stdout = NULL, stderr = NULL)  
  }  
  Sys.sleep(0.5)
```

- 確保沒有舊的 chromedriver 佔用 port
- Windows 用 taskkill /F /IM; Linux/Mac 用 pkill

start_driver()：在背景啟動 chromedriver

```
system2(driver_path,  
        args = sprintf("--port=%d", port),  
        wait = FALSE,  
        stdout = NULL, stderr = NULL)  
Sys.sleep(2)
```

- wait = FALSE：**不等** chromedriver 結束就繼續執行
 - chromedriver 是持續運行的伺服器，不會自行結束
 - 若 wait = TRUE，R 會永遠卡在這一行
- Sys.sleep(2)：給 chromedriver 2 秒準備接受連線

start_driver(): Chrome 啟動參數

```
chrome_args <- c(  
  "--no-sandbox",  
  "--disable-dev-shm-usage",  
  "--window-size=1920,1080",  
  "--disable-blink-features=AutomationControlled",  
  if (headless) "--headless=new"  
)
```

參數	用途
--no-sandbox	Linux 必要：停用安全沙盒
--disable-dev-shm-usage	避免共享記憶體不足
--window-size	設定虛擬視窗大小
--disable-blink-features=...	降低被偵測為機器人
--headless=new	不顯示視窗（排程用）

start_driver(): 建立 Session (W3C 格式)

```
body <- list(capabilities = list(
  alwaysMatch = list(
    browserName = "chrome",
    "goog:chromeOptions" = list(
      args = as.list(chrome_args))))

res <- POST(
  sprintf("http://localhost:%d/session", port),
  body = body, encode = "json")
val <- fromJSON(rawToChar(res$content))
sess <- list(id = val$value$sessionId,
            port = port)
if (is.null(sess$id) || is.na(sess$id))
  stop("WebDriver session 建立失敗:",
       toJSON(val, auto_unbox = TRUE))
sess
}
```

- goog: 前綴 = Google 定義的非標準擴充欄位

stop_driver(): 關閉瀏覽器

```
stop_driver <- function(sess) {  
  tryCatch(DELETE(sprintf(  
    "http://localhost:%d/session/%s",  
    sess$port, sess$id)),  
    error = function(e) NULL)  
  if (.Platform$OS.type == "windows") {  
    system2("taskkill",  
      args = c("/F", "/IM",  
        "chromedriver.exe"),  
      wait = FALSE,  
      stdout = NULL, stderr = NULL)  
  } else {  
    system2("pkill", args = "chromedriver",  
      wait = FALSE,  
      stdout = NULL, stderr = NULL)  
  }  
  invisible(NULL)  
}
```

習題：start/stop_driver()

- ❶ `system2(..., wait = FALSE)` 啟動 `chromedriver` 時為什麼必要？
- ❷ `stop_driver` 中的 `tryCatch` 有什麼作用？
- ❸ 何時用 `headless = TRUE`？何時用 `FALSE`？

解答：start/stop_driver()

- ❶ chromedriver 是持續運行的伺服器；wait = TRUE 會讓 R 永遠等待它結束而卡住。
- ❷ 若 Chrome 已崩潰，DELETE 會出錯。tryCatch 捕獲錯誤，確保後續的 pkill 仍會執行。
- ❸ TRUE：排程自動執行（不顯示視窗）。FALSE：課堂示範（學生看得到操作過程）。

thsr.R 逐段解析：wd_js 與 js_str

wd_js(): 在瀏覽器裡執行 JavaScript

```
wd_js <- function(sess, script,
                  args = list()) {
  body_json <- sprintf(
    '{"script":%s,"args":%s}',
    toJSON(script, auto_unbox = TRUE),
    toJSON(args))
  url <- paste0(.wd_base(sess),
               "/execute/sync")
  res <- POST(url, body = body_json,
             add_headers(
               "Content-Type" = "application/json"))
  fromJSON(rawToChar(res$content))$value
}
```

這是整個程式**最核心**的函式。

wd_js(): 為什麼手動組 JSON

R 的 `list()` 序列化行為：

```
toJSON(list())           # → "{}"   ← JSON 物件  
toJSON(character(0))     # → "[]"   ← JSON 陣列
```

WebDriver 規範要求 `args` 是 **JSON 陣列** `[]`。用 `httr` 的 `encode = "json"` 自動處理整個 `body` 時，嵌套的 `list()` 可能被錯誤序列化。

解法：用 `toJSON()` 分別序列化 `script` 和 `args`，再用 `sprintf` 手動拼成 JSON 字串，完全控制格式。

wd_js() : return 的重要性

```
# 有 return → R 收到 TRUE 或 FALSE
```

```
wd_js(sess,  
  'return document.querySelector(  
    "#idNumber") !== null;')
```

```
# 沒有 return → R 收到 NULL
```

```
wd_js(sess,  
  'document.querySelector(  
    "#idNumber") !== null;')
```

- /execute/sync 只回傳 JavaScript 明確 return 的值
- 忘記 return 是最常見的錯誤

js_str(): 安全的字串引號

```
js_str <- function(x)
  toJSON(x, auto_unbox = TRUE)
```

```
js_str("#SubmitButton")
# → '"#SubmitButton"' ← 含雙引號
```

用於 sprintf 中安全插入 JS:

```
sprintf(
  "document.querySelector(%s).click();",
  js_str("#SubmitButton"))
# → 'document.querySelector(
#   "#SubmitButton").click();'
```

- toJSON 自動加引號並處理特殊字元跳脫

習題：wd_js() 與 js_str()

- ❶ 為什麼 wd_js 不直接用 POST(..., encode = "json")？
- ❷ wd_js(sess, 'return 1 + 2;') 的回傳值是什麼？
- ❸ wd_js(sess, 'document.title;') 有什麼問題？
- ❹ js_str("hello world") 的輸出？

解答：wd_js() 與 js_str()

- ❶ 避免 `list()` 被序列化為 JSON 物件 `{}` 而非陣列 `[]`。手動組 JSON 可精確控制格式。
- ❷ 3 (JavaScript 的 $1 + 2 = 3$ ，透過 `return` 回傳給 R)。
- ❸ 缺少 `return`，R 收到 NULL。應改為 `'return document.title;'`。
- ❹ `"hello world"` (含雙引號的字串，可安全插入 JavaScript 程式碼)。

thsr.R 逐段解析：DOM 工具函式

wd_navigate(): 前往網址

```
wd_navigate <- function(sess, url) {  
  wd_cmd(sess, "POST", "/url", list(url = url))  
  invisible(NULL)  
}
```

對應 HTTP 請求：

```
POST http://localhost:4444/session/7db2.../url  
{ "url": "https://irs.thsrc.com.tw/..." }
```

- 透過 wd_cmd 發送 POST，路徑 /url
- list(url = url) 被 encode = "json" 轉成 JSON

wait_for(): 輪詢等待元素出現

```
wait_for <- function(sess, css,
  timeout = WAIT_TIMEOUT,
  poll = POLL_FREQ) {
  deadline <- Sys.time() + timeout
  script <- sprintf(
    "return document.querySelector('%s') !== null;",
    js_str(css))
  repeat {
    found <- tryCatch(
      isTRUE(wd_js(sess, script)),
      error = function(e) FALSE)
    if (found) return(invisible(TRUE))
    if (Sys.time() > deadline)
      stop(sprintf(" 等待逾時: %s", css))
    Sys.sleep(poll)
  }
}
```

wait_for(): 輪詢模式圖解

動態網頁最重要的模式 — **輪詢等待**：

t=0.0s: `querySelector("#idNumber")` → `false`

t=0.4s: 再問 → `false`

t=0.8s: 再問 → `false`

... (頁面還在載入)

t=2.4s: 再問 → `true`

- 不能假設元素「現在就在」 — 頁面載入需要時間
- `POLL_FREQ = 0.4` 秒問一次
- `WAIT_TIMEOUT = 20` 秒超時 → `stop()` 出錯
- `tryCatch`: 連線錯誤也當作「未找到」

wait_click(): 等待後點擊

```
wait_click <- function(sess, css,
                        timeout = WAIT_TIMEOUT) {
  wait_for(sess, css, timeout)
  wd_js(sess, sprintf(
    "document.querySelector(%s).click();",
    js_str(css)))
  invisible(NULL)
}
```

- 先等元素出現 (wait_for)，再用 JS 的 .click() 點擊
- 用 JS 而非 WebDriver 的 Element Click 端點：只需一次 HTTP 請求，更簡潔

習題：wait_for() 與 wait_click()

- ❶ wait_for 的 repeat {...} 有幾種結束方式？
- ❷ 如果網路很慢，WAIT_TIMEOUT = 20 不夠，怎麼辦？
- ❸ 為什麼 wait_click 不直接 .click() 而要先 wait_for？
- ❹ isTRUE(wd_js(sess, script)) 中 isTRUE() 的作用？

解答：wait_for() 與 wait_click()

- ❶ 兩種：(a) 元素出現 → `return(invisible(TRUE))`；(b) 超時 → `stop()` 拋出錯誤。
- ❷ 呼叫時傳入更大的 `timeout` 參數，如 `wait_for(sess, css, timeout = 60)`。
- ❸ 元素可能尚未載入，`querySelector` 回傳 `null`，對 `null` 呼叫 `.click()` 會出錯。
- ❹ `wd_js` 可能回傳 `NULL`、`NA` 或其他非布林值。`isTRUE()` 確保只有嚴格的 `TRUE` 才通過。

fill()：在輸入框填入文字

```
fill <- function(sess, css, text) {  
  wait_for(sess, css)  
  wd_js(sess, sprintf('  
    (function() {  
      var el = document.querySelector(%s);  
      el.focus();  
      el.value = "";  
      el.value = %s;  
      el.dispatchEvent(  
        new Event("input", {bubbles:true}));  
      el.dispatchEvent(  
        new Event("change", {bubbles:true}));  
    })());  
    ', js_str(css), js_str(text)))  
  invisible(NULL)  
}
```


- ❶ **IIFE** — `(function() { ... })()`
 - 「立即執行函式」，建立獨立作用域
 - 避免 `var el` 汙染全域命名空間
- ❷ **先清空再填值** — `el.value = ""; el.value = %s;`
 - 確保前端框架 (React、Vue) 的變更偵測正確觸發
 - 若直接設值，框架可能認為「值沒變」而不更新
- ❸ **同時觸發 `input` + `change`**
 - 不同網站監聽不同事件
 - 兩個都觸發確保相容

js_select(): 設定下拉選單

```
js_select <- function(sess, css, value) {  
  wait_for(sess, css)  
  wd_js(sess, sprintf('  
    (function() {  
      var el = document.querySelector(%s);  
      el.value = %s;  
      el.dispatchEvent(  
        new Event("change", {bubbles:true}));  
    })());  
' , js_str(css), js_str(value)))  
  invisible(NULL)  
}
```

- 與 fill() 類似，但只觸發 change（下拉選單不需要 input 和 focus）
- 就是你在 Console 手動測試過的那段 JavaScript

習題：fill() 與 js_select()

- ❶ fill() 和 js_select() 的主要差異？
- ❷ fill() 為什麼要呼叫 el.focus()？
- ❸ fill(sess, "#idNumber", "A123456789") 做了什麼？
- ❹ dispatchEvent 若少了 {bubbles: true} 會怎樣？

解答：fill() 與 js_select()

- ❶ fill() 觸發 input + change、先清空再填值、呼叫 focus()。js_select() 只觸發 change，直接設值。
- ❷ 模擬使用者「點進」輸入框，某些網站的驗證邏輯依賴 focus 事件。
- ❸ 等 #idNumber 出現 → 聚焦 → 清空 → 填入 "A123456789" → 觸發 input 和 change。
- ❹ 事件不會向上冒泡給父元素，父層的監聯器收不到事件，可能導致頁面連動失效。

click_link_text(): 依文字內容點擊連結

```
click_link_text <- function(sess, text,
  timeout = WAIT_TIMEOUT) {
  deadline <- Sys.time() + timeout
  script <- sprintf('
    (function() {
      var a = Array.from(
        document.querySelectorAll("a"))
        .find(function(el) {
          return el.textContent.trim()
            === %s;
        });
      if (a) { a.click(); return true; }
      return false;
    })();
  ', js_str(text))
```

- `querySelectorAll("a")`: 找**所有** `<a>` 標籤
- `Array.from(...)`: `NodeList` → 真正的陣列 (才有 `.find()`)

click_link_text()：輪詢直到點擊成功

```
# 接上頁
repeat {
  ok <- tryCatch(
    isTRUE(wd_js(sess, script)),
    error = function(e) FALSE)
  if (ok) return(invisible(TRUE))
  if (Sys.time() > deadline)
    stop(sprintf(
      " 等待逾時：連結「%s」 ", text))
  Sys.sleep(POLL_FREQ)
}
}
```

- textContent.trim()：取元素文字，去前後空白
- ===：JavaScript 嚴格相等（型別 + 值都要一致）
- 用於沒有穩定 id / class 的連結，只能靠文字定位

- ❶ `querySelectorAll` 和 `querySelector` 的差異？
- ❷ 為什麼要用 `Array.from()` 包裝 `querySelectorAll` 的結果？
- ❸ JavaScript 的 `===` 和 `==` 有何不同？
- ❹ 這個函式在 `thsr.R` 中哪裡被用到？為什麼需要它？

解答：click_link_text()

- ❶ `querySelector` 回傳**第一個**符合的元素；`querySelectorAll` 回傳**所有**符合的元素（`NodeList`）。
- ❷ `NodeList` 沒有 `.find()` 方法。`Array.from()` 轉成真正的 `Array` 才能使用陣列方法。
- ❸ `===` 嚴格相等（型別也要一致）；`==` 會自動型別轉換（如 `"1" == 1` 為 `true`）。JS 建議用 `===`。
- ❹ 用在 `cancel()` 中點擊「取消訂位」連結，因為該連結沒有穩定的 `id`，只能靠文字內容定位。

thsr.R 逐段解析：截圖與驗證碼

element_screenshot(): 等待圖片載入

```
element_screenshot <- function(sess, css,
  filename = "captcha.png") {
  wait_for(sess, css)
  wd_js(sess, sprintf('
    (function() {
      var el = document.querySelector(%s);
      if (el && el.tagName === "IMG"
        && !el.complete) {
        return new Promise(function(resolve) {
          el.addEventListener("load",
            function() { resolve(true); });
          el.addEventListener("error",
            function() { resolve(false); });
          setTimeout(function() {
            resolve(false); }, 5000);
        });
      }
      return true;
    })()
  ', js_str(css)))
  Sys.sleep(0.3)
```

上頁的 JavaScript 使用了 **Promise**：

- 如果元素是 `<img>` 且尚未載入完成 (`!el.complete`)
 - 回傳一個 Promise，等待 `load` 或 `error` 事件
 - 5 秒超時保底
- 如果已載入或不是 `<img>` → 直接回傳 `true`

WebDriver 的 `/execute/sync` 端點會自動等待 **Promise resolve**，所以 R 端不需要額外處理非同步邏輯。

element_screenshot(): 取得位置與 DPR

```
rect <- wd_js(sess, sprintf('
  var dpr = window.devicePixelRatio || 1;
  var r = document.querySelector(%s)
    .getBoundingClientRect();
  return {
    x: r.left * dpr, y: r.top * dpr,
    width: r.width * dpr,
    height: r.height * dpr, dpr: dpr};
', js_str(css)))
```

- **DPR (Device Pixel Ratio)** = 螢幕像素倍率
 - 普通螢幕 DPR = 1 ; Retina / 高解析度 DPR = 2
 - 截圖的實際像素數 = CSS 像素 × DPR
- `getBoundingClientRect()` 回傳 CSS 像素 → 需乘以 DPR 才對準截圖

element_screenshot(): 截圖裁切存檔

```
raw <- wd_cmd(sess, "GET", "/screenshot")
img <- image_read(base64decode(raw))
img <- image_crop(img,
  sprintf("%dx%d+%d+%d",
    round(rect$width), round(rect$height),
    round(rect$x),      round(rect$y)))
image_write(img, filename)
invisible(filename)
}
```

- GET /screenshot: 全頁截圖, base64 編碼
- base64decode(): base64 字串 → 二進位
- image_read(): magick 讀入圖片
- image_crop(" 寬 x 高 +X+Y"): 裁切出驗證碼區域
- image_write(): 存成 PNG 檔案

```
page_screenshot <- function(sess, filename) {  
  raw <- wd_cmd(sess, "GET", "/screenshot")  
  writeBin(base64decode(raw), filename)  
  invisible(filename)  
}
```

- 比 element_screenshot 簡單：不需裁切，整頁存檔
- writeBin()：將二進位資料直接寫入檔案
- 用於訂票成功 / 失敗時的截圖存證

- ❶ Base64 編碼的用途是什麼？為什麼不直接傳二進位圖片？
- ❷ `image_crop("100x40+950+600")` 代表什麼？
- ❸ $DPR = 2$ 時，CSS 寬度 `100px` 的元素在截圖中佔幾個像素？
- ❹ `element_screenshot` 為什麼要先等圖片載入完成？

- ❶ 截圖是二進位 PNG，但 JSON 只能傳文字。Base64 把二進位編碼成純文字（英數字 + 少數符號），讓 JSON 可以承載。
- ❷ 從座標 (950, 600) 開始，裁出寬 100、高 40 像素的區域。
- ❸ **200 像素** (CSS 100px × DPR 2 = 200 實際像素)。
- ❹ 如果 尚未載入完成就截圖，可能截到空白或破損的圖片，導致驗證碼無法辨識。

ocr_captcha(): 手動驗證碼辨識

```
ocr_captcha <- function(  
  filename = "captcha.png",  
  manual = FALSE) {  
  if (manual) {  
    if (interactive()) {  
      tryCatch(print(image_scale(  
        image_read(filename), "400")),  
        error = function(e) NULL)  
    }  
    cat(sprintf(  
      "\n[驗證碼] 請查看 %s，輸入後按 Enter: ",  
      filename))  
    code <- trimws(readline())  
    return(gsub("[^a-zA-Z0-9]", "", code))  
  }  
  ""  
}
```

- `interactive()`：是否在 RStudio Console 等互動環境
 - `TRUE` → 用 `image_scale` 放大圖片並顯示
 - `FALSE` (Rscript 排程) → 不嘗試顯示
- `image_scale(..., "400")`：放大到 400px 寬以利肉眼辨識
- `readline()`：暫停等使用者輸入後按 Enter
- `gsub("[^a-zA-Z0-9]", "", code)`：去除非英數字符
 - `[^...]` = 「不在集合裡的字符」
- `manual = FALSE`：回傳空字串（預留未來自動辨識）

- ❶ `gsub("[^a-zA-Z0-9]", "", " Ab-3! ")` 的結果？
- ❷ 為什麼用 `tryCatch` 包住 `print(image_scale(...))`？
- ❸ `manual = FALSE` 回傳空字串，驗證碼怎麼通過？

解答：ocr_captcha()

- ❶ "Ab3" (只保留英文字母和數字，其餘全部去除)。
- ❷ 在無圖形環境 (SSH 遠端、無 X11) 中，`print(image_...)` 會出錯。`tryCatch` 捕獲錯誤，確保不影響主程式。
- ❸ 空字串送出後驗證碼必然失敗，進入 `while` 迴圈重試。這是預留給未來自動辨識 (如 Python dddocr) 的接口。

thsr.R 逐段解析：book() 訂票主函式

book(): 函式簽名與初始化

```
book <- function(date,
                  headless = FALSE,
                  manual = TRUE) {
  sess <- start_driver(headless)
  on.exit(stop_driver(sess), add = TRUE)

  wd_navigate(sess, THSR_BOOK_URL)
  wait_for(sess, "#BookingS1Form",
            timeout = 30)
  tryCatch(
    wait_click(sess, "#cookieAccpetBtn",
               timeout = 5),
    error = function(e) NULL)
```

- `on.exit(..., add = TRUE)`: 不管正常結束或出錯，一定關閉 Chrome
- Cookie 按鈕用 `tryCatch`: 首次出現並點擊；之後不出現則靜默略過

book() : reached() 判斷函式

```
reached <- function()  
  tryCatch(isTRUE(wd_js(sess,  
    'return document.querySelector(  
      "#idNumber") != null;')),  
    error = function(e) FALSE)  
  
trials <- 0L  
while (!reached() && trials < NUM_TRIALS) {  
  trials <- trials + 1L  
  message(sprintf("  驗證碼 #%d", trials))  
}
```

- reached() : 檢查是否已進入旅客資料頁 (第二頁)
- 判斷指標：#idNumber (身分證欄位) 只在第二頁出現
- while 迴圈：驗證碼錯就重試，最多 NUM_TRIALS 次

book(): 填寫出發站、到達站、日期

```
tryCatch({
  js_select(sess,
    "#BookingS1Form_seatCon_seatRadioGroup",
    SEAT)
  js_select(sess,
    "[name='selectStartStation']",
    start_sta)
  js_select(sess,
    "[name='selectDestinationStation']",
    dest_sta)
  wd_js(sess, sprintf('
    var el = document.querySelector(
      "#toTimeInputField");
    el.value = %s;
    el.dispatchEvent(new Event("change",
      {bubbles: true}));
  ', js_str(date)))
```


book(): 車次選擇與驗證碼

```
wait_click(sess,  
  "input[data-target='search-by-trainNo']")  
fill(sess,  
  "[name='toTrainIDInputField']",  
  train_id)  
element_screenshot(sess,  
  "#BookingS1Form_homeCaptcha_passCode",  
  "captcha.png")  
code <- ocr_captcha("captcha.png",  
                    manual = manual)  
message(sprintf("    OCR: '%s'", code))  
fill(sess,  
  "[name='homeCaptcha:securityCode']",  
  code)  
Sys.sleep(0.5)  
wait_click(sess, "#SubmitButton")
```

book()：驗證碼重試邏輯

```
tryCatch(  
  wait_for(sess, "#idNumber",  
            timeout = 8),  
  error = function(e) {  
    message("    未通過，重試...")  
    wd_navigate(sess, THSR_BOOK_URL)  
    wait_for(sess, "#BookingS1Form",  
              timeout = 30)  
    tryCatch(wait_click(sess,  
                        "#cookieAccpetBtn", timeout = 5),  
              error = function(e2) NULL)  
  }  
)  
, error = function(e)  
  message(sprintf("    例外: %s",  
                  conditionMessage(e))))  
} # while 結束
```

- 提交後等 8 秒看 #idNumber 是否出現
- 未出現 → 驗證碼錯誤 → 回到訂票首頁重來

book(): 驗證碼失敗處理

```
if (!reached()) {  
    message(sprintf(  
        " 訂票失敗：%d 次皆未成功",  
        NUM_TRIALS))  
    page_screenshot(sess, "thsr_fail.png")  
    return(FALSE)  
}
```

- 超過最大嘗試次數 → 截圖存證 → 回傳 FALSE
- page_screenshot 記錄失敗時的頁面狀態

book(): 填寫旅客資料

```
message(" 通過！填寫旅客資料...")
fill(sess, "#idNumber", PID)
fill(sess, "#mobilePhone", MOBILE)
tryCatch(
  fill(sess, paste0(
    "#BookingS3Form_",
    "TicketPassengerInfoInputPanel_",
    "passengerDataView_0_",
    "passengerDataView2_",
    "passengerDataIdNumber"), PID),
  error = function(e)
    message(" 不須輸入乘客資訊"))
tryCatch({
  wait_click(sess, "#memberSystemRadio1",
    timeout = 3)
  fill(sess, "#msNumber", MEMBER_PID)
}, error = function(e) NULL)
```

book()：完成訂位與截圖

```
wait_click(sess, "[name='agree']")
wait_click(sess, "#isSubmit")
tryCatch({
  wait_for(sess, "[name='SubmitButton']",
            timeout = 8)
  wait_click(sess, "[name='SubmitButton']")
}, error = function(e) NULL)
Sys.sleep(2)
page_screenshot(sess,
  sprintf("thsr_ok_%s.png",
          gsub("/", "", date)))
message(" 訂票成功！截圖已存。")
TRUE
}
```

- 勾選同意 → 送出 → 確認頁再送出
- 某些情況無確認頁 → tryCatch 靜默略過
- 成功截圖：thsr_ok_20260413.png

- ❶ `on.exit(stop_driver(sess), add = TRUE)` 的作用？若函式中途出錯，Chrome 會關閉嗎？
- ❷ 為什麼用 `while (!reached() && trials < NUM_TRIALS)` 而非 `for`？
- ❸ 填寫乘客身分證的 `tryCatch` 只 `message` 不 `stop`，為什麼？
- ❹ 最後 `tryCatch` 包住 `wait_for(sess, "[name='SubmitButton']")` 的原因？

- ❶ 函式結束（含出錯）時一定執行 `stop_driver(sess)` 關閉 Chrome。**會關閉**，`on.exit` 就是為此設計。
- ❷ 迴圈不只看次數，還檢查 `reached()`：一旦成功進入第二頁就立即結束，不浪費嘗試。
- ❸ 某些票種（如兒童票）不要求輸入乘客身分證，元素不存在時略過即可，**不應中斷**整個訂票流程。
- ❹ 某些情況下沒有確認頁（直接訂票成功），`[name='SubmitButton']` 不存在，超時後靜默略過。

thsr.R 逐段解析：cancel() 取消訂位

cancel(): 函式簽名與初始化

```
cancel <- function(order_id,
                    headless = FALSE,
                    manual = TRUE) {
  sess <- start_driver(headless)
  on.exit(stop_driver(sess), add = TRUE)

  wd_navigate(sess, THSR_CANCEL_URL)
  wait_for(sess, "#rocId", timeout = 30)
```

- 結構和 book() 相似
- 導航至**取消訂位頁面**
- 等待 #rocId (身分證輸入欄) 出現

```
reached <- function()  
  tryCatch(isTRUE(wd_js(sess,  
    'return document.querySelector(  
      ".btn-edit") !== null;')),  
    error = function(e) FALSE)  
  
trials <- 0L  
while (!reached() && trials < NUM_TRIALS) {  
  trials <- trials + 1L  
  message(sprintf("  驗證碼 #%d", trials))  
  fill(sess, "#rocId", PID)  
  fill(sess, "#orderId", order_id)
```

- 成功指標：.btn-edit（訂位列表中的編輯按鈕）
- 每次重試都需重新填寫身分證和訂位代號

cancel()：驗證碼與提交

```
element_screenshot(sess,
  "#HistoryForm_divCaptcha_passCode",
  "captcha.png")
code <- ocr_captcha("captcha.png",
  manual = manual)
message(sprintf("    OCR: '%s'", code))
fill(sess, "#securityCode", code)
Sys.sleep(0.5)
wait_click(sess, "[name='SubmitButton']")
tryCatch(
  wait_for(sess, ".btn-edit", timeout = 5),
  error = function(e)
    message("    未通過"))
}

if (!reached()) {
  message(sprintf(
    "  取消失敗：%d 次皆未成功", NUM_TRIALS))
  page_screenshot(sess,
    "thsr_cancel_fail.png")
  return(FALSE)
```

cancel()：關閉查詢彈窗

```
message(" 通過！開始取消訂位...")
tryCatch({
  wait_for(sess,
    "div.uk-modal.uk-open", timeout = 3)
  wd_js(sess, '
    var btn = document.querySelector(
      "div.uk-modal.uk-open \
input.uk-modal-close");
    if (btn) btn.click();
  ')
  Sys.sleep(1)
}, error = function(e)
  message(" 無查詢彈窗，略過"))
```

- 部分情境會彈出 Modal 視窗
- CSS Selector `div.uk-modal.uk-open`：找已開啟的 Modal
- 點擊 Modal 內的關閉按鈕
- 不一定出現 → `tryCatch` 靜默略過

cancel()：點擊「取消訂位」連結

```
wait_click(sess, ".btn-edit")
wait_for(sess,
    'a[href*="Cancel"]', a', timeout = 10)
Sys.sleep(0.5)
wd_js(sess, '
(function() {
    var links = Array.from(
        document.querySelectorAll("a"));
    var target = links.find(function(a) {
        return a.textContent.trim()
            === "\u53d6\u6d88\u8a02\u4f4d";
    });
    if (target) {
        target.click(); return true; }
    return false;
})();
')
```

"\u53d6\u6d88\u8a02\u4f4d" = 「取消訂位」

Unicode	字
\u53d6	取
\u6d88	消
\u8a02	訂
\u4f4d	位

為什麼用 Unicode Escape ?

- R 用 `sprintf` 將字串插入 JavaScript 時，中文可能因編碼轉換 (UTF-8 / CP950 / Big5) 出錯
- `\uXXXX` 是純 ASCII，完全迴避編碼風險

cancel()：完成取消

```
    Sys.sleep(0.5)
    wait_click(sess, "[name='agree']")
    wait_click(sess, "[name='SubmitButton']")
    Sys.sleep(3)
    message(sprintf(" 訂位 %s 取消完成",
                    order_id))
    invisible(TRUE)
}
```

- 勾選同意 → 點擊送出
- Sys.sleep(3)：等伺服器處理完成
- invisible(TRUE)：回傳 TRUE 但不自動印出

- ❶ cancel() 用 .btn-edit 判斷成功，book() 用 #idNumber，為什麼不同？
- ❷ 為什麼「取消訂位」連結不用 CSS Selector 而用文字搜尋？
- ❸ 如果中文直接寫在 JavaScript 字串中（不用 \uXXXX），可能出什麼問題？

解答：cancel()

- ❶ 頁面結構不同：訂票成功 → 旅客資料頁（有 `#idNumber`）；取消查詢成功 → 訂位列表頁（有 `.btn-edit`）。
- ❷ 該連結沒有穩定的 `id` 或 `class`，只能透過 `textContent` 定位。
- ❸ R 的 `sprintf` 在不同系統上的字串編碼不一致（UTF-8 vs CP950 等），中文可能變亂碼。`\uXXXX` 是純 ASCII，不受系統編碼影響。

thsr.R 逐段解析：範例呼叫

```
if (TRUE) {  
  book(date = "2026/04/13")  
  # cancel(order_id = "01901879")  
}
```

- `book(date = "2026/04/13")`：訂 4/13 的車票
- `cancel(...)` 已註解：需要時取消註解即可
- `if (TRUE)`：方便快速改成 `FALSE` 停用

預設參數：`headless = FALSE`（顯示 Chrome）、`manual = TRUE`（手動輸入驗證碼）。

執行方式

```
source("thsr.R")  
  
# headless = FALSE：顯示 Chrome（學生看到操作）  
# manual = TRUE：手動輸入驗證碼  
book(date = "2026/04/13",  
      headless = FALSE, manual = TRUE)
```

執行流程：

- ❶ Chrome 自動開啟並填寫出發站、到達站、日期、車次
- ❷ 截取驗證碼 → 在 RStudio Viewer 顯示放大圖
- ❸ Console 提示：[驗證碼] 請查看 captcha.png，輸入後按 Enter：
- ❹ 成功後截圖存為 thsr_ok_20260413.png

```
# 訂位代號在訂票截圖或簡訊中 (8 碼英數字)
cancel(order_id = "01901879",
        headless = FALSE, manual = TRUE)
```

```
source("thsr.R")
sess <- start_driver(headless = FALSE)

wd_navigate(sess, THSR_BOOK_URL)
wd_js(sess, 'return document.title;')
# → " 台灣高鐵網路訂票"

wd_js(sess, 'return document.querySelector(
  "#BookingS1Form") !== null;')
# → TRUE

js_select(sess,
  "[name='selectStartStation']", "11")
# 觀察：Chrome 的下拉選單變成「台南」

page_screenshot(sess, "demo.png")
stop_driver(sess)
```

附錄

代碼	車站	代碼	車站
1	南港	7	台中
2	台北	8	彰化
3	板橋	9	雲林
4	桃園	10	嘉義
5	新竹	11	台南
6	苗栗	12	左營

在 DevTools 中查看 `<select name="selectStartStation">` 的 `<option>` 即可得到這些代碼。

Selector	元素
#cookieAccpetBtn	Cookie「我同意」按鈕
#BookingS1Form	訂票主表單
[name='selectStartStation']	出發站下拉選單
[name='selectDestinationStation']	到達站下拉選單
#toTimeInputField	出發日期欄
input[data-target='search-by-trainNo']	「車次查詢」radio
[name='toTrainIDInputField']	車次號碼輸入欄
#BookingS1Form_homeCaptcha_passCode	驗證碼圖片
[name='homeCaptcha:securityCode']	驗證碼輸入欄
#SubmitButton	「開始查詢」按鈕

Selector	元素
#idNumber	旅客身分證號 (訂票第二頁)
#mobilePhone	旅客手機號碼
[name='agree']	同意條款 checkbox
#isSubmit	「完成訂位」按鈕
#memberSystemRadio1	TGo 會員選項
#msNumber	TGo 會員號碼輸入欄
#rocId	取消訂位：身分證欄
#orderId	取消訂位：訂位代號欄
#HistoryForm_divCaptcha_passCode	取消訂位：驗證碼圖片
.btn-edit	訂位列表的操作按鈕